

CORRIGÉ DU DEVOIR À LA MAISON 16

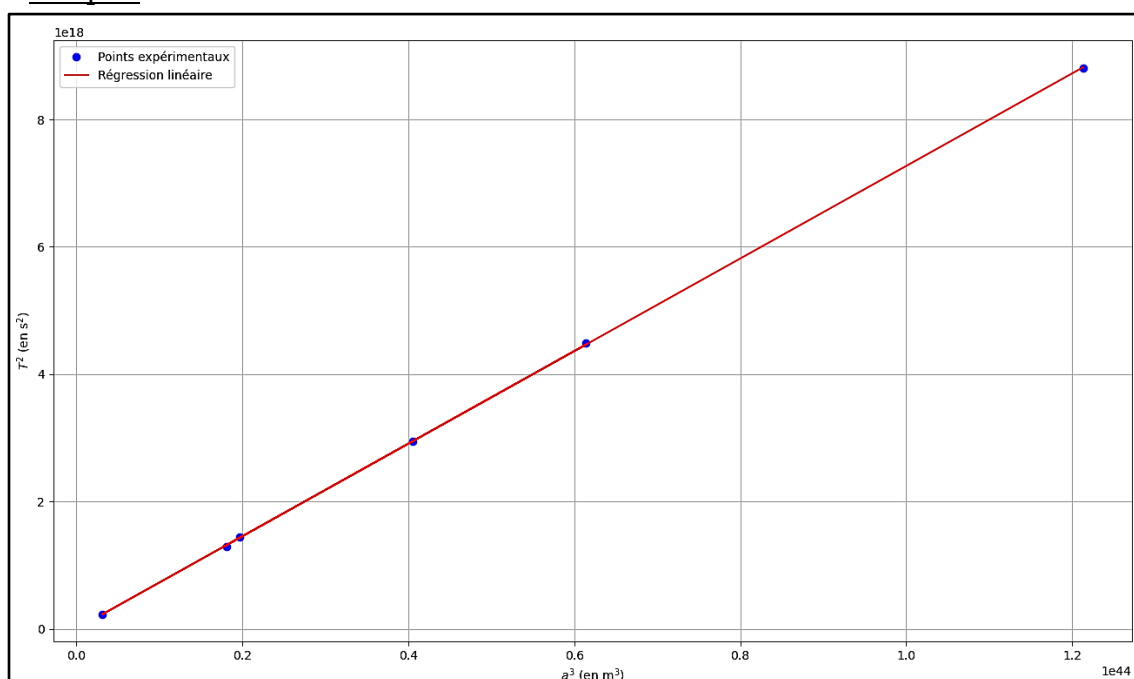
Vérification de la 3^{ème} loi de Kepler

❖ Simulation numérique en Python : Vérification de la 3^{ème} loi de Kepler

Astuce Python : Pour effectuer la même opération mathématique sur chaque élément d'un tableau, il suffit d'effectuer cette opération directement sur le tableau ! C'est le cas ici pour les conversions d'unités, et l'élévation au carré ou au cube...

Commentaire : on vérifie la loi de Kepler car la régression linéaire de T^2 en fonction de a^3 passe par les points expérimentaux

➤ Graphe



❖ Simulation numérique en Python : Masse du trou noir

Loi de Kepler : $\frac{T^2}{a^3} = \frac{4\pi^2}{GM}$ soit $M = \frac{4\pi^2}{G} \frac{a^3}{T^2}$

Masse du trou noir (obtenue par étude statistique) : $M = \frac{4\pi^2}{G} \frac{1}{K}$

Incertitude-type composée : $\frac{u(M)}{M} = \frac{u(K)}{K} \Leftrightarrow u(M) = M \frac{u(K)}{K}$

Valeurs numériques : $M = 8,1581 \cdot 10^{36} \text{ kg}$ et $u(M) = 0,0307 \cdot 10^{36} \text{ kg}$

➤ Écart normalisé : $EN = \frac{|M - M_t|}{\sqrt{u^2(M) + u^2(M_t)}} = 2,3$: de l'ordre de 2, donc l'estimation est correcte.

➤ Script Python :

```

1  """Exécuter les cellules les unes après les autres après les avoir complétées
2  en cliquant sur Run puis Execute Cell (ou CTRL + Entrée)
3  Pour exécuter l'ensemble du fichier : CTRL + E
4  """
5  # 3ème loi de Kepler
6
7  ## Cellule 1 : Importation des bibliothèques utiles
8  import numpy as np          # pour la manipulation des tableaux
9  import matplotlib.pyplot as plt  # pour les représentations graphiques
10
11  ## Cellule 2 : Vérification de la 3ème loi de Kepler
12  tabT = np.array([94.1,15.24,67.2,54.4,36.,38])*365*24*3600  # Tableau des périodes T en secondes!
13  taba = np.array([3300.,980.,2630.,2290.,1750.,1800.])*1.5*10**11 # Tableau des demi-grand axes a en
    mètres!
14  x = taba**3                # Tableau des abscisses a^3
15  y = tabT**2                # Tableau des ordonnées T^2
16  """
17  p = np.polyfit(x, y, n)
18  Modélise la courbe y = f(x) par un polynôme de degré n
19  Arguments:
20      x : tableau des abscisses
21      y : tableau des ordonnées
22      n : degré du polynôme (pour n = 1 : régression linéaire)
23  Renvoie:
24      p : tableau des coefficients du polynôme tel que :
25          p[0] : coefficient de degré n, p[1] : coefficient de degré n-1...
26          p[n] : coefficient de degré 0
27  """
28
29  p = np.polyfit(x, y, 1) # Tableau des coefficients de la régression linéaire de y en fonction de x
30  ymodele = p[0]*x + p[1] # Equation de la régression linéaire ymodele en fonction de x
31
32  plt.figure(figsize = (16,9))
33  plt.plot(x, y, 'bo', label="Points expérimentaux") # Tracé des points expérimentaux avec des ronds
    bleus
34  plt.plot(x, ymodele, 'r-', label="Régression linéaire") # Tracé de la régression linéaire en trait
    rouge
35  plt.xlabel(r"$a^3$ (en m$^3$)")
36  plt.ylabel(r"$T^2$ (en s$^2$)")
37  plt.grid()
38  plt.legend(loc = 'upper left')
39  plt.show()
40
41  ## Cellule 3 : Masse du trou noir
42  tabK = tabT**2 / taba**3    # Tableau des valeurs T^2 / a^3
43  K = np.mean(tabK)           # Valeur moyenne
44  sK = np.std(tabK,ddof=1)    # Ecart-type expérimental
45  uK = sK / np.sqrt(len(tabK)) # Incertitude-type
46
47  G = 6.67*1e-11 # Constante de gravitation
48  M = 4 * np.pi**2 / (G*K)  # Expression de la masse M
49  uM = M * uK / K            # Expression de l'incertitude-type uM
50  print(f"Masse du trou noir : M = {M:.4e} kg") #Affichage de la masse du trou noir
51  print(f"Incertitude-type : u(M) = {uM:.2e} kg ") #Affichage de l'incertitude-type
52
53  ## Cellule 4 : Ecart normalisé
54  Mt = 8.2541*1e36           # Valeur tabulée de M
55  uMt = 0.0278*1e36          # Valeur tabulée de uM
56  EN = abs(M - Mt) / np.sqrt(uM**2 + uMt**2) # Calcul de l'écart normalisé
57  print(f"Ecart normalisé : EN = {EN:.1f}") #Affichage de l'écart normalisé

```

➤ Affichage des résultats

```

Masse du trou noir : M = 8.1581e+36 kg
Incertitude-type : u(M) = 3.07e+34 kg
Ecart normalisé : EN = 2.3

```